The commands that once looked like alien text — `chmod 755`, `tail -f`, `grep -i`, `awk '{print $1}'`, `tar -czf`, `ssh-keygen`, `crontab -e` — you now understand all of them. You know when to use them, why to use them, and what happens when things go wrong.

That is the difference between someone who memorized commands and someone who actually understands Linux. You are in the second category now.

Keep practicing every day. Consistency is what gets you the job.

Now go update your document with Day 13 to Day 16 and you will have the complete Linux reference for life! 💪

Say **"update document"** and I will pack everything from Day 13 to Day 16 into your notes!

yes , update the document and start Git and Github First day

Let me do both simultaneously — generate the Git Day 1 content AND build the updated complete document.

Git & GitHub — Day 1: Introduction & Core Concepts

Why Git is Non-Negotiable for DevOps

Every single company that writes code uses Git. No exceptions. When you join a DevOps team your first task will involve Git — cloning a repository, making changes, pushing code, reviewing pull requests.

Git is also how DevOps connects to everything else:

- CI/CD pipelines trigger when you push to Git
- Docker images are built from code stored in Git
- Infrastructure as Code (Terraform, Ansible) lives in Git
- Kubernetes manifests live in Git

No Git knowledge = cannot work in any modern tech company.

What You Will Learn Today

- What is Git and why it exists
- Git vs GitHub — the difference
- Installing and configuring Git

- The three areas of Git
- Your first repository
- Core workflow — init, add, commit, status, log
- Writing good commit messages
- The `.gitignore` file

What is Git?

Git is a **version control system** — it tracks every change made to your files over time.

Imagine you are writing a deployment script. Without Git:

- You save `deploy.sh`
- You make changes — something breaks
- You cannot go back to the working version
- You have no idea what changed

With Git:

- Every version is saved with a message explaining what changed
- You can go back to any previous version instantly
- You can see exactly what changed between versions
- Multiple people can work on the same files without overwriting each other

Think of Git like a time machine for your code.

What is GitHub?

```
Git ≠ GitHub

Git     = the tool installed on your computer that tracks changes
GitHub  = a website that hosts your Git repositories online
```

Git	GitHub
Runs locally on your machine	Runs in the cloud — website
Tracks changes in files	Stores repositories online
Created by Linus Torvalds 2005	Created by GitHub Inc 2008
Free and open source	Free tier + paid plans
Works without internet	Requires internet

Other alternatives to GitHub: **GitLab** (self-hostable, popular in enterprises), **Bitbucket** (Atlassian — common in corporate environments).

Installing Git

```bash
# Ubuntu/Debian:
sudo apt update
sudo apt install -y git

# Verify installation:
git --version
# Output: git version 2.39.0
```

Configuring Git — Do This First

Before using Git you must tell it who you are. This information is attached to every commit you make.

```bash
git config --global user.name "Varun"
git config --global user.email "varun@email.com"
git config --global core.editor "vim"
git config --global init.defaultBranch "main"

# Verify your config:
git config --list
```

`--global` means this applies to all repositories on your machine. Without `--global

---

The Three Areas of Git — Most Important Concept

This is what most beginners do not understand and it causes all the confusion.

```
Working Directory    →    Staging Area    →    Repository
(your files)             (git add)             (git commit)

where you edit       where you prepare    where history
your files           your changes         is saved forever
```

Think of it like packing for a trip:

- **Working Directory** = your room with all your stuff
- **Staging Area** = your suitcase where you decide what to pack
- **Repository** = the suitcase is closed and ready — history saved

You choose exactly what goes into each commit using the staging area. You might change 10 files but only commit 3 of them together.

Your First Repository

Creating a New Repository

```bash
cd ~/linux_practice
mkdir git_practice
cd git_practice

# Initialize git repository
git init
```

Output:
```
Initialized empty Git repository in /home/varun/git_practice/.git/
```

This creates a hidden `.git` folder — Git stores everything here. Never manually edit or delete this folder.

```bash
ls -la    # see the .git folder
```

The `.git` Folder

```bash
ls .git/
```
```
HEAD        # points to current branch
config      # repository-specific config
objects/    # all your file content stored here
```

```
refs/       # branches and tags
hooks/      # scripts that run on git events
```

You never need to touch these directly — Git manages them. But knowing they exist helps when debugging.

Core Git Commands

`git status` — Your Most Used Command

```bash
git status
```

Shows:

- Which branch you are on
- Which files are modified
- Which files are staged (ready to commit)
- Which files are untracked (Git does not know about them)

Run `git status` constantly. Before every add. After every add. Before every commit. It tells you exactly what is happening.

`git add` — Moving to Staging Area

```bash
# Create a file first
echo "My first DevOps script" > deploy.sh
echo "Server configuration" > config.yml

git status    # shows both files as "untracked"

git add deploy.sh           # add one specific file
git add config.yml          # add another file
git add .                   # add ALL changed files in current folder
git add -p                  # interactive — choose which parts to stage
```

⚠️ `git add .` Warning

```bash
```

```bash
git add .    # adds everything — including files you did not mean to add
```

Always run `git status` after `git add .` to see exactly what was staged. Better habit — add files individually or use `.gitignore` first.

`git commit` — Saving to History

```bash
git commit -m "Add deployment script and config file"
```

`-m` flag adds the message inline. Without `-m` Git opens your configured editor.

```bash
# Stage and commit in one command (only for tracked files):
git commit -am "Fix typo in deploy script"
# -a stages all MODIFIED tracked files automatically
# Does NOT add new untracked files — use git add for those
```

Writing Good Commit Messages — Critical

Bad commit messages:
```
"fix"
"changes"
"update"
"asdfgh"
"WIP"
```

Good commit messages:
```
"Add nginx configuration for production server"
"Fix disk space check in monitoring script"
"Update database backup to run at 1am daily"
"Remove hardcoded password from deploy script"
```

Format: Start with a verb. Be specific. Keep under 72 characters. Explain WHAT and WHY — not HOW.

Interviewers often ask to show your Git history. Bad commit messages show unprofessional habits.

`git log` — Viewing History

```bash
git log                    # full history — press q to quit
git log --oneline          # one line per commit — cleaner
git log --oneline -10      # last 10 commits only
git log --oneline --graph # shows branch graph
git log --author="Varun"  # commits by specific person
git log --since="7 days ago"  # commits in last 7 days
git log -p                 # shows actual changes in each commit
```

Each commit has:

```
commit a1b2c3d4e5f6...     # SHA — unique identifier for this commit
Author: Varun <varun@email.com>
Date:   Thu Mar 20 10:30:00 2024

    Add deployment script and config file
```

The SHA (like `a1b2c3d`) is how you reference any commit. You only need first 7 characters.

`git diff` — See What Changed

```bash
git diff                   # changes in working directory not yet staged
git diff --staged          # changes staged but not yet committed
git diff HEAD              # all changes since last commit
git diff a1b2c3 d4e5f6     # difference between two commits
```

The `.gitignore` File

What is .gitignore?

A file that tells Git which files and folders to completely ignore — never track, never add, never show in `git status`.

```bash
vim .gitignore
```

What to Put in .gitignore

bash

```bash
# Secrets and environment files — NEVER commit these
.env
.env.local
*.pem
*.key
secrets.yml

# Logs — no need to track
*.log
logs/

# OS generated files
.DS_Store          # Mac
Thumbs.db          # Windows
desktop.ini

# Dependencies — too large, regenerated from package files
node_modules/
vendor/
__pycache__/
*.pyc

# Build output
dist/
build/
target/
*.class
*.jar

# IDE files
.vscode/
.idea/
*.swp          # Vim swap files

# Terraform
*.tfstate
*.tfstate.backup
.terraform/
```

Creating .gitignore for Your Practice

bash

```bash
cd ~/linux_practice/git_practice
```

```bash
cat > .gitignore << 'EOF'
*.log
.env
*.pem
*.key
.DS_Store
__pycache__/
*.pyc
EOF

git add .gitignore
git commit -m "Add gitignore for logs, secrets and OS files"
```

Global .gitignore — Applies to All Repos

```bash
vim ~/.gitignore_global

# Add OS files:
.DS_Store
Thumbs.db
*.swp

git config --global core.excludesfile ~/.gitignore_global
```

Now these are ignored in every repository automatically.

Complete First Day Workflow

The Exact Steps — Memorize This

```bash
# 1. Initialize repository
git init

# 2. Check status — always first
git status

# 3. Create .gitignore before anything else
vim .gitignore

# 4. Add files to staging
git add .
```

```bash
# 5. Check what was staged
git status

# 6. Commit with clear message
git commit -m "Initial commit — add project files"

# 7. Verify commit was saved
git log --oneline
```

Practice — Do This Right Now

```bash
cd ~/linux_practice/git_practice

# Create some files
echo "#!/bin/bash" > deploy.sh
echo "echo 'Deploying application...'" >> deploy.sh
echo "DB_HOST=localhost" > .env
echo "server_name localhost;" > nginx.conf

# Initialize and make first commit
git init
cat > .gitignore << 'EOF'
.env
*.log
*.pem
EOF

git add .gitignore
git commit -m "Add gitignore"

git add deploy.sh nginx.conf
git commit -m "Add deployment script and nginx config"

git log --oneline
git status
```

Notice `.env` does not appear in `git status` — `.gitignore` is working.

Summary — Git Day 1

Command	What it does
`git config --global user.name`	Set your name

Command	What it does
`git init`	Initialize new repository
`git status`	See current state — use constantly
`git add filename`	Stage specific file
`git add .`	Stage all changes
`git commit -m "message"`	Save staged changes to history
`git log --oneline`	View commit history
`git diff`	See unstaged changes
`git diff --staged`	See staged changes

Interview Questions — Git Day 1

Q1. What is Git and why is it used? Answer: Git is a distributed version control system that tracks changes to files over time. Used to collaborate on code, maintain history of all changes, revert to previous versions and work on features in parallel using branches. Every DevOps team uses Git.

Q2. What is the difference between Git and GitHub? Answer: Git is the version control tool installed locally on your machine. GitHub is a cloud platform that hosts Git repositories online for collaboration and backup. GitHub also provides features like pull requests, issues and CI/CD pipelines.

Q3. What are the three areas of Git? Answer: Working Directory where you edit files, Staging Area where you prepare changes with `git add`, and Repository where history is permanently saved with `git commit`.

Q4. What is the difference between `git add .` and `git add -p`? Answer: `git add .` stages all changed files at once. `git add -p` goes through each change interactively asking which parts to stage — useful when you want to commit only specific changes from a file.

Q5. What is a commit SHA? Answer: A unique 40-character hash identifier automatically generated for every commit. Used to reference specific commits — you only need first 7 characters. Example: `git diff a1b2c3d e4f5g6h` compares two commits.

Q6. Why should .env files never be committed to Git? Answer: .env files contain secrets — database passwords, API keys, access tokens. Committing them exposes these secrets to anyone with repository access. On public repositories this is a serious security incident. Always add .env to .gitignore.

Homework — Before Git Day 2

1. Configure Git with your name and email

2. Create a new folder, initialize a repository
3. Create 3 files — a script, a config and a readme
4. Add a proper `.gitignore`
5. Make 3 separate commits with clear messages
6. Run `git log --oneline` and see your history
7. Edit a file then run `git diff` before staging

Git Day 2 we learn **Branches, Merging and Resolving Conflicts** — the most important Git concept for team collaboration. Say **"Git Day 2"** whenever you are ready! 💪

Now let me build the complete updated document with everything Day 1 to Day 16 plus Git Day 1.

Good — I'll build the new document extending the existing one with Days 13-16 and Git Day 1.

Your complete updated document is ready! 📄

What's packed inside

Section	Content
Progress Tracker	All 16 Linux days + Git days with status
Linux Days 1–12	Full master command reference table
Linux Day 13	SSH, scp, rsync, key pairs, config file, errors
Linux Day 14	Vim modes, navigation, editing, search/replace
Linux Day 15	Environment variables, PATH, .bashrc, .env files
Linux Day 16	Interview mega revision — scenarios, quick fire, phrases
Git Day 1	Git vs GitHub, three areas, init, add, commit, log, .gitignore
Master Reference	Git Day 1 command table

Your Current Progress

```
Linux  ━━━━━━━━━━━━━━━━━━━━━━━━  16/16 ✅ COMPLETE
Git    ━━━━━━━━━━━━━━━━━━━━━━━  1/4  🔥 Started
```

Git Roadmap Ahead

Day	Topic
Git Day 1 ✅	Init, add, commit, log, .gitignore
Git Day 2	Branches, merging, conflict resolution
Git Day 3	GitHub, remote, push, pull, pull requests
Git Day 4	Advanced — rebase, stash, cherry-pick, tags

Say **"Git Day 2"** whenever you are ready! 💪

Linux git complete notes
Document · DOCX
Download

Day 2

Git Day 2 — Branches, Merging & Conflict Resolution

Why This is Critical for DevOps

Imagine your team is working on a live production website. One developer is fixing a critical bug. Another is building a new feature. A third is updating the database schema.

If everyone works on the same code at the same time — they overwrite each other's changes. Everything breaks.

Branches solve this. Every developer works in their own isolated copy. When ready — they merge back. This is how every real company works.

Branching and merging is also the **most asked Git topic in DevOps interviews.**

What You Will Learn Today

- What a branch is
- Creating and switching branches
- The main/master branch
- Merging branches
- Fast-forward vs three-way merge

- Merge conflicts — what they are and how to fix them
- Deleting branches
- Real DevOps branching strategies
- Branch naming conventions

What is a Branch?

A branch is an **independent line of development.** It is a pointer to a specific commit. When you create a branch you get your own copy of the codebase to work in — without affecting anyone else.

```
main branch:    A --- B --- C
                              \
feature branch:                D --- E --- F
```

Main branch continues normally. Feature branch has its own commits. When feature is done — merge F back into main.

What is HEAD?

`HEAD` is a pointer that shows **where you currently are** in the repository — which branch and which commit you are on.

```bash
cat .git/HEAD
# ref: refs/heads/main
# This means you are on the main branch
```

When you switch branches — HEAD moves. When you make a commit — HEAD moves forward.

Creating and Switching Branches

Basic Branch Commands

```bash
git branch                    # list all local branches
git branch -a                 # list all branches including remote
git branch -v                 # list branches with last commit message

git branch feature-login      # create new branch
git switch feature-login      # switch to that branch
```

```
git switch main                # switch back to main

# Create AND switch in one command — most common:
git switch -c feature-login    # modern way — recommended
git checkout -b feature-login # older way — still works, same result
```

`git switch` vs `git checkout`

```bash
git switch branch-name         # modern — only for switching branches
git checkout branch-name       # older — works but does more things
```

`git switch` was introduced in Git 2.23 (2019) specifically for switching branches. `git checkout` is older and does multiple things — switching branches, restoring files, checking out commits. Both work but `git switch` is clearer and recommended for switching branches.

Seeing Which Branch You Are On

```bash
git branch                     # current branch has * in front
git status                     # first line always shows current branch
```

Output of `git branch`:

```
* feature-login
  main
  bugfix-navbar
```

The `*` shows you are on `feature-login`.

Practice — Create Your First Branch

```bash
cd ~/linux_practice/git_practice

# Make sure you have some commits first
git log --oneline

# Create and switch to feature branch
git switch -c feature-monitoring
```

```bash
# Verify you are on the new branch
git branch
git status

# Make a change on this branch
echo "#!/bin/bash" > monitor.sh
echo "systemctl is-active nginx || sudo systemctl start nginx" >> monitor.sh

git add monitor.sh
git commit -m "Add nginx monitoring script"

# Your main branch does NOT have this file yet
git switch main
ls    # monitor.sh is NOT here — it only exists on feature-monitoring

git switch feature-monitoring
ls    # monitor.sh IS here — back on feature branch
```

This demonstrates that branches are truly isolated. Your changes on one branch do not exist on others until you merge.

Merging Branches

What is Merging?

Merging takes the changes from one branch and combines them into another branch.

```bash
# Standard workflow:
# 1. Finish work on feature branch
# 2. Switch to main branch
# 3. Merge feature branch into main
# 4. Delete feature branch

git switch main
git merge feature-monitoring
```

Two Types of Merges

Type 1 — Fast-Forward Merge

```
Happens when main has not moved since you created your branch. Git simply moves the
```
Before merge:
main: A --- B
                 \
feature: C --- D

After fast-forward merge:
main: A --- B --- C --- D
```

No merge commit created. Clean, linear history.

```bash
git switch main
git merge feature-monitoring
# Output: Fast-forward
```

Type 2 — Three-Way Merge

```
Happens when both branches have new commits since the branch was created. Git create
```
Before merge:
main: A --- B --- C
                 \
feature: D --- E

After three-way merge:
main: A --- B --- C --- M
                 \   /
feature: D --- E
```

`M` is the merge commit — it has two parents.

```bash
git switch main
git merge feature-login
# Output: Merge made by the 'recursive' strategy.
# A new merge commit is created automatically
```

Merge Commit Message

When a three-way merge happens Git opens your editor asking for a merge commit message. Default message is fine — just save and quit:

```bash
# In vim — just type :wq to accept default message
# In nano — Ctrl+X then Y then Enter
```

Merge Conflicts — The Most Important Part

What is a Merge Conflict?

A conflict happens when **both branches changed the same part of the same file.** Gi

```
main branch:    line 5 = "port 8080"
feature branch: line 5 = "port 9090"

When you merge — Git says: CONFLICT — which one do you want?
```

When Conflicts Happen

```bash
git switch main
git merge feature-branch
# Output:
# Auto-merging config.yml
# CONFLICT (content): Merge conflict in config.yml
# Automatic merge failed; fix conflicts and then commit the result.
```

What a Conflict Looks Like Inside the File

Open the conflicting file:

```bash
cat config.yml
```

```
server_name = "myapp"
<<<<<<< HEAD
```

```
port = 8080
=======
port = 9090
>>>>>>> feature-branch
database_host = "localhost"
```

Part	Meaning
`<<<<<<< HEAD`	Start of YOUR version (current branch)
`=======`	Divider between two versions
`>>>>>>> feature-branch`	End of THEIR version (branch being merged)

Resolving a Conflict — Step by Step

Step 1 — Open the conflicting file

```bash
vim config.yml
```

Step 2 — Decide which version to keep

You have three choices:

- Keep your version (HEAD)
- Keep their version (feature-branch)
- Keep both / write something new

Edit the file to remove the conflict markers and leave only the final version:

```bash
# Remove all conflict markers and leave what you want:
server_name = "myapp"
port = 9090
database_host = "localhost"
```

Step 3 — Stage the resolved file

```bash
git add config.yml
```

Step 4 — Complete the merge

```bash
```

```bash
git commit -m "Merge feature-branch — use port 9090"
```

Checking Conflict Status

```bash
git status     # shows files with conflicts under "both modified"
git diff       # shows conflict markers in files
```

Aborting a Merge

If conflicts are too complex and you want to start over:

```bash
git merge --abort    # cancels merge and restores previous state
```

Practice — Create and Resolve a Conflict

```bash
cd ~/linux_practice/git_practice

# Setup — create a file on main
git switch main
echo "port=8080" > app.conf
git add app.conf
git commit -m "Add app config with port 8080"

# Create branch and change same line
git switch -c fix-port
sed -i 's/port=8080/port=9090/' app.conf
git add app.conf
git commit -m "Change port to 9090"

# Switch to main and also change same line differently
git switch main
sed -i 's/port=8080/port=7777/' app.conf
git add app.conf
git commit -m "Change port to 7777"

# Now try to merge — CONFLICT will happen
git merge fix-port

# See the conflict
```

```bash
cat app.conf

# Resolve it — edit file, keep 9090
echo "port=9090" > app.conf
git add app.conf
git commit -m "Resolve port conflict — use 9090"

git log --oneline --graph
```

Deleting Branches

Cleaning Up After Merge

```bash
git branch -d feature-monitoring    # delete merged branch — safe
git branch -D feature-monitoring    # force delete — even if not merged
```

`-d` (lowercase) only deletes if the branch has been fully merged. It protects you from accidentally deleting unmerged work.

`-D` (uppercase) force deletes regardless. Use only when you are sure you do not need the branch.

Best Practice — Always Delete After Merging

```bash
# Full workflow:
git switch main
git merge feature-login
git branch -d feature-login    # clean up immediately
git log --oneline --graph       # verify clean history
```

Undeleted branches pile up and make `git branch` output messy. Delete them as soon a

---

Real DevOps Branching Strategies

Strategy 1 — GitHub Flow (Most Common for DevOps)

Simple and practical. Used by most modern teams:

```
main branch       = always deployable — production code
feature branches  = all new work happens here
```

```bash
# Workflow:
git switch -c feature/add-monitoring    # create feature branch
# ... do work, make commits ...
git switch main
git merge feature/add-monitoring        # merge when done
git branch -d feature/add-monitoring    # clean up
git push origin main                    # deploy to production
```

Strategy 2 — GitFlow (Larger Teams)

More structured — used in companies with scheduled releases:

```
main        = production code — only release commits here
develop     = integration branch — all features merge here first
feature/*   = individual features
release/*   = release preparation
hotfix/*    = emergency production fixes
```

```bash
# Feature work:
git switch -c feature/new-dashboard develop
# ... work ...
git switch develop
git merge feature/new-dashboard

# Release:
git switch -c release/v1.2 develop
# ... testing, fixes ...
git switch main
git merge release/v1.2
git tag v1.2
```

For a DevOps engineer starting out — **GitHub Flow is what you will use and be asked

---

Strategy 3 — Trunk Based Development

```
Everyone commits directly to main (trunk) frequently. Feature flags control what use
```
```
main = everyone pushes here daily
# Features hidden with feature flags until ready
```

Used by large companies like Google and Facebook. Requires very good CI/CD pipeline and testing.

Branch Naming Conventions

Type	Format	Example
Feature	`feature/description`	`feature/add-monitoring`
Bug fix	`bugfix/description`	`bugfix/fix-login-error`
Hotfix	`hotfix/description`	`hotfix/critical-security-patch`
Release	`release/version`	`release/v1.2.0`
Experiment	`experiment/description`	`experiment/new-auth-system`

Always use lowercase and hyphens. No spaces. No special characters.

Useful Branch Commands

Seeing Branch History

```bash
git log --oneline --graph --all    # see ALL branches and their commits
git log --oneline main..feature    # commits in feature not in main
git log --oneline feature..main    # commits in main not in feature
```

Comparing Branches

```bash
git diff main feature-branch           # all differences between branches
git diff main..feature-branch          # same with .. notation
git diff main feature-branch -- file.txt  # differences in specific file
```

Checking if Branch is Merged

```bash
git branch --merged        # branches already merged into current branch
git branch --no-merged     # branches NOT yet merged — safe to check before delete
```

Renaming a Branch

```bash
git branch -m old-name new-name        # rename branch
git branch -m new-name                 # rename CURRENT branch
```

Full Summary — Git Day 2

Command	What it does
`git branch`	List all branches
`git branch name`	Create new branch
`git switch name`	Switch to branch
`git switch -c name`	Create AND switch in one step
`git merge branch`	Merge branch into current
`git merge --abort`	Cancel merge in progress
`git branch -d name`	Delete merged branch safely
`git branch -D name`	Force delete branch
`git log --graph --all`	See all branches visually
`git diff main..feature`	Compare two branches
`git branch --merged`	Show merged branches
`git branch --no-merged`	Show unmerged branches

Interview Questions — Git Day 2

Q1. What is a Git branch and why is it used? Answer: A branch is an independent line of development — a pointer to a specific commit. Used so multiple developers can work on

different features simultaneously without affecting each other's code. Each developer works in isolation and merges when ready.

Q2. What is the difference between fast-forward and three-way merge? Answer: Fast-forward happens when the target branch has not moved since the feature branch was created — Git simply moves the pointer forward, no merge commit created, clean linear history. Three-way merge happens when both branches have new commits — Git creates a new merge commit combining both histories.

Q3. What is a merge conflict and how do you resolve it? Answer: A conflict occurs when both branches changed the same part of the same file. Git marks the conflicting section with conflict markers. To resolve: open the file, remove the markers and edit to keep the correct version, then git add the file and git commit to complete the merge.

Q4. What is HEAD in Git? Answer: HEAD is a pointer showing where you currently are in the repository — which branch and commit you are on. When you switch branches HEAD moves. When you commit HEAD moves forward. `cat .git/HEAD` shows where HEAD points.

Q5. What is the difference between `git branch -d` and `git branch -D`? Answer: -d (lowercase) safely deletes a branch only if it has been fully merged — protects against losing work. -D (uppercase) force deletes regardless of merge status. Always use -d first and only use -D when certain the work is not needed.

Q6. Explain GitHub Flow branching strategy. Answer: Main branch always contains deployable production code. All new work happens on feature branches. When feature is complete it is merged into main and the feature branch is deleted. Simple, practical and used by most modern DevOps teams.

Q7. What happens to your changes when you switch branches? Answer: If changes are committed they stay on their branch and disappear when you switch. If changes are not committed Git either carries them over or refuses to switch if there is a conflict — preventing data loss.

Q8. How do you see the commit history across all branches? Answer: `git log --oneline --graph --all` — shows all branches and their commits in a visual graph format.

Homework — Before Git Day 3

```bash
cd ~/linux_practice/git_practice
```

1. Create a branch called `feature/disk-monitor`
2. Add a script `disk_monitor.sh` with content from Day 9
3. Commit it with a good message
4. Switch back to main and create another branch `feature/log-cleaner`
5. Add a script `log_cleaner.sh` that deletes logs older than 30 days

6. Commit it
7. Merge `feature/disk-monitor` into main
8. Merge `feature/log-cleaner` into main
9. Delete both feature branches
10. Run `git log --oneline --graph` — see your merge history

Git Day 3 we learn **GitHub, Remote Repositories, Push, Pull and Pull Requests** — this is where Git goes from local to team collaboration. Every DevOps workflow depends on this. 💪

Say **"Git Day 3"** whenever you are ready!

Git Day 3

Git Day 3 — GitHub, Remote Repositories, Push, Pull & Pull Requests

---

Why This is Critical for DevOps

Everything you have done so far in Git has been local — only on your machine. In real DevOps work:

- Your code lives on GitHub — not just your laptop
- CI/CD pipelines trigger when you push to GitHub
- Your team reviews your code through Pull Requests before it goes to production
- Other engineers pull your changes to stay in sync
- Docker images are built from code hosted on GitHub

GitHub is where Git meets the real world of team collaboration and automation.

What You Will Learn Today

- Creating a GitHub account and repository
- Connecting local repo to GitHub (remote)
- `git push` — sending your commits to GitHub
- `git pull` — getting changes from GitHub
- `git fetch` — checking changes without merging

- `git clone` — downloading a repository
- Pull Requests — the heart of team collaboration
- Forking repositories
- Real DevOps GitHub workflows

Quick Concept — Local vs Remote

```
LOCAL                           REMOTE (GitHub)
(your machine)                  (cloud)

git_practice/                   github.com/varun/git_practice
├── .git/                       ├── main branch
├── deploy.sh                   ├── feature branches
└── nginx.conf                  └── pull requests

git push  ──────────────────>   sends your commits up
git pull  <──────────────────   brings their commits down
```

Setting Up GitHub

Step 1 — Create GitHub Account

Go to github.com and create a free account. Use your professional email — this account will be on your resume.

Step 2 — Set Up SSH Authentication with GitHub

GitHub no longer accepts passwords for git operations. You must use SSH keys or Personal Access Tokens. SSH keys are cleaner.

```bash
# Generate key (if you don't have one already):
ssh-keygen -t ed25519 -C "varun@email.com"

# Copy your PUBLIC key:
cat ~/.ssh/id_ed25519.pub
```

Go to GitHub → Settings → SSH and GPG Keys → New SSH Key → paste your public key → Save.

Test the connection:

```bash
```

```bash
ssh -T git@github.com
# Output: Hi varun! You've successfully authenticated
```

Step 3 — Create Repository on GitHub

1. Click `+` → New repository
2. Repository name: `devops-scripts`
3. Description: "Linux and DevOps automation scripts"
4. Choose Public or Private
5. **Do NOT initialize with README** — you have local code already
6. Click Create repository

GitHub shows you the commands to connect — we will use SSH URL format.

Connecting Local Repo to GitHub

What is a Remote?

A remote is a **named link to a repository on another server.** By convention the main remote is called `origin`.

```bash
# Add remote to your local repo:
git remote add origin git@github.com:varun/devops-scripts.git

# Verify remote was added:
git remote -v

# Output:
# origin  git@github.com:varun/devops-scripts.git (fetch)
# origin  git@github.com:varun/devops-scripts.git (push)
```

Remote Commands

```bash
git remote -v                          # list all remotes
git remote add origin URL              # add new remote
git remote remove origin               # remove remote
```

```bash
git remote rename origin upstream      # rename remote
git remote set-url origin NEW_URL      # change remote URL
```

`git push` — Sending to GitHub

Basic Push

```bash
git push origin main
```

This pushes your `main` branch to `origin` (GitHub).

First Push — Setting Upstream

The first time you push a branch you must tell Git where to push it:

```bash
git push -u origin main
# -u sets upstream — remembers where to push/pull
# After this you can just type: git push
```

`-u` or `--set-upstream` links your local branch to the remote branch. After running this once — just type `git push` and it knows where to go.

Pushing Different Branches

```bash
git push origin main              # push main branch
git push origin feature-login     # push feature branch
git push -u origin feature-login  # push and set upstream
git push                          # push current branch (after -u set)
git push origin --all             # push ALL branches at once
```

Pushing Tags

```bash
git push origin v1.0              # push specific tag
git push origin --tags            # push all tags
```

⚠️ Force Push — Use With Extreme Caution

```bash
git push --force origin main      # overwrites remote history
git push --force-with-lease       # safer force push — checks for others' changes
```

Force push rewrites the remote branch history. If someone else has pulled that branch — their history now conflicts. **Never force push to main/master in a team environment.** Only acceptable on your own feature branches when rebasing.

`git clone` — Downloading a Repository

What is Cloning?

Cloning creates a complete local copy of a remote repository — all files, all history, all branches.

```bash
git clone git@github.com:varun/devops-scripts.git
# Creates a folder called devops-scripts with everything inside

git clone git@github.com:varun/devops-scripts.git my-folder
# Clone into a specific folder name

git clone --depth 1 git@github.com:varun/devops-scripts.git
# Shallow clone — only latest commit, no history — faster
```

Clone vs Init + Remote Add

```bash
# Use clone when:
# → You want to work on an existing project
git clone git@github.com:company/project.git

# Use init + remote add when:
# → You have existing local code you want to push to new GitHub repo
git init
git remote add origin git@github.com:varun/project.git
```

What Clone Creates

```bash
git clone git@github.com:varun/devops-scripts.git
cd devops-scripts

git remote -v      # origin already set automatically
git branch -a      # shows all remote branches too
git log --oneline  # full history is there
```

Cloning automatically sets `origin` to point back to GitHub. No manual remote add needed.

`git fetch` VS `git pull`

The Difference — Very Important for Interviews

```bash
git fetch origin          # downloads changes but does NOT merge
git pull origin main      # downloads AND merges into current branch
```

Think of it like checking your mailbox:

- `git fetch` = go to mailbox, see what arrived, bring it inside but don't open yet
- `git pull` = go to mailbox, bring it inside AND read and act on everything

---

`git fetch` — Safe Way to Check

```bash
git fetch origin              # download all remote changes
git log --oneline HEAD..origin/main   # see what changed on remote
git diff HEAD origin/main     # see exact differences
git merge origin/main         # NOW merge if you are happy
```

`git fetch` is safer because it lets you inspect what changed before integrating. In production environments many engineers prefer fetch + review + merge over a direct pull.

`git pull` — Fetch + Merge in One

```bash
git pull origin main          # fetch and merge main
git pull                      # pull current branch (after upstream set)
```

```bash
git pull --rebase             # fetch and rebase instead of merge
```

`git pull --rebase` keeps history cleaner — instead of creating a merge commit it replays your commits on top of the fetched commits. Many teams require this.

When to Use Which

Situation	Use
Want to see what changed before merging	`git fetch` then review
Quick update on your own branch	`git pull`
Team requires clean history	`git pull --rebase`
Before starting new work	`git fetch origin`

Tracking Remote Branches

Seeing Remote Branches

```bash
git branch -a                # all branches including remote
git branch -r                # remote branches only
```

Output:

```
* main
  feature-login
  remotes/origin/main
  remotes/origin/feature-monitoring
```

`remotes/origin/main` = the state of main on GitHub as of last fetch.

Working with Remote Branches

```bash
# Get a remote branch that does not exist locally:
git switch -c feature-login origin/feature-login
# or shorter:
git switch feature-login    # Git automatically tracks remote branch

# See tracking relationships:
```

```
git branch -vv
# Output shows which local branch tracks which remote branch
```

Pull Requests — The Heart of Collaboration

What is a Pull Request?

A Pull Request (PR) is a request to merge your branch into another branch — with a *
```
Your laptop          GitHub               Team
feature branch  →   Open PR         →   Review code
                     ↓                   ↓
                    CI runs tests        Approve / Request changes
                     ↓                   ↓
                    Merge to main    →   Code deployed
```

PRs are how professional teams:

- Review code before it goes to production
- Discuss changes and suggest improvements
- Trigger automated testing (CI/CD)
- Maintain code quality

Creating a Pull Request — Step by Step

Step 1 — Create and push feature branch:

bash

```bash
git switch -c feature/add-backup-script
echo "#!/bin/bash" > backup.sh
echo "tar -czf /backup/home_$(date +%Y%m%d).tar.gz /home/varun/" >> backup.sh
git add backup.sh
git commit -m "Add automated home directory backup script"
git push -u origin feature/add-backup-script
```

Step 2 — Open PR on GitHub:
- Go to your repository on GitHub
- GitHub shows a yellow banner: "feature/add-backup-script had recent pushes"
- Click "Compare & pull request"
- Add title: "Add automated home directory backup script"

- Add description: explain what it does and why
- Click "Create pull request"

Step 3 — Review and Merge:
- Team members review the code
- They can add comments on specific lines
- They approve or request changes
- Once approved — click "Merge pull request"
- Click "Delete branch" — clean up

Writing a Good PR Description

Bad description:
```
Fixed the thing
```

Good description:
```
## What this does
Adds an automated backup script that creates a compressed tar archive
of the home directory daily.

## Why
Currently we have no automated backups. This prevents data loss.

## How to test
Run ./backup.sh manually — verify a .tar.gz file appears in /backup/

## Related
Closes #45 — feature request for automated backups
```

Good PR descriptions get approved faster and show professional maturity.

PR Review Best Practices

As the reviewer:
```
- Check the code does what the description says
- Look for security issues (hardcoded passwords, wrong permissions)
- Check error handling — what happens when it fails?
- Verify naming is clear and consistent
- Test it yourself if possible
```

```
As the author:
```
- Keep PRs small — one feature or fix per PR
- Respond to all comments
- Never merge your own PR if team policy requires review
- Update the PR if changes are requested
```

---

# Forking — Contributing to Other Projects

---

## What is Forking?

Forking creates **your own copy of someone else's repository** on GitHub. Used when
```
Original repo: github.com/company/project
Your fork: github.com/varun/project   (your copy)
Your local clone: ~/project   (on your machine)
```

---

### Fork Workflow

bash

```bash
# 1. Fork on GitHub — click Fork button on the repo page

# 2. Clone YOUR fork:
git clone git@github.com:varun/project.git

# 3. Add original repo as upstream:
git remote add upstream git@github.com:company/project.git

# 4. Verify remotes:
git remote -v
# origin    git@github.com:varun/project.git (your fork)
# upstream  git@github.com:company/project.git (original)

# 5. Create feature branch and make changes:
git switch -c fix-typo-in-docs

# 6. Push to YOUR fork:
git push origin fix-typo-in-docs
```
```

```
# 7. Open PR from your fork to original repo on GitHub
```

Keeping Your Fork Updated

```bash
git fetch upstream                  # get changes from original
git switch main
git merge upstream/main             # update your main
git push origin main                # update your fork on GitHub
```

Real DevOps GitHub Workflows

Workflow 1 — Solo DevOps Engineer

```bash
# Daily workflow:
git pull                              # start with latest code
git switch -c feature/add-monitoring  # new branch for each task
# ... make changes ...
git add .
git commit -m "Add disk monitoring cron job"
git push -u origin feature/add-monitoring
# Open PR on GitHub → merge → delete branch
git switch main
git pull                              # get the merged changes
git branch -d feature/add-monitoring  # clean up local
```

Workflow 2 — Team Environment

```bash
# Before starting work:
git fetch origin
git log --oneline HEAD..origin/main   # see what teammates pushed
git pull --rebase                     # update with clean history

# Do your work:
git switch -c bugfix/fix-deploy-script
# ... make fixes ...
git add .
git commit -m "Fix deploy script — handle missing env variable"
```

```bash
git push -u origin bugfix/fix-deploy-script

# Open PR → get review → address feedback → merge
```

Workflow 3 — Hotfix on Production

```bash
# Critical bug in production — urgent fix needed
git switch main
git pull                              # get latest production code
git switch -c hotfix/fix-auth-crash

# Fix the bug
vim auth.sh
git add auth.sh
git commit -m "Fix auth crash when DB_PASSWORD not set"

git push -u origin hotfix/fix-auth-crash
# Open PR — get emergency review
# Merge immediately
# Deploy to production

git switch main
git pull
git branch -d hotfix/fix-auth-crash
```

Workflow 4 — CI/CD Trigger on Push

This is how GitHub Actions CI/CD works:

```yaml
# .github/workflows/deploy.yml
on:
  push:
    branches: [main]    # triggers when code pushed to main

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: ./deploy.sh
```

Every time you merge a PR to main — GitHub automatically runs your deployment script. This is the core of DevOps automation. You will set this up fully when you reach CI/CD in your roadmap.

Common GitHub Situations and Solutions

Situation 1 — Remote Has Changes You Don't Have

```bash
git push origin main
# Error: rejected — remote contains work not in local

# Fix:
git pull --rebase origin main   # bring in remote changes first
git push origin main            # now push works
```

Situation 2 — Wrong Commit on Wrong Branch

```bash
# You committed to main instead of feature branch
git log --oneline    # find the commit SHA — e.g. a1b2c3d

# Create feature branch with that commit
git switch -c feature/my-change

# Remove commit from main (move pointer back)
git switch main
git reset HEAD~1     # undo last commit — keeps changes in working dir
```

Situation 3 — Need to Update PR with New Changes

```bash
# Reviewer requested changes on your PR
git switch feature/add-monitoring    # go back to feature branch
# ... make the requested changes ...
git add .
git commit -m "Address review feedback — add error handling"
git push origin feature/add-monitoring    # PR updates automatically
```

Situation 4 — Sync Feature Branch with Latest Main

```bash
# Main has moved while you were working on feature branch
git switch main
git pull                          # update main
git switch feature/my-feature
git merge main                    # bring main changes into feature
# or
git rebase main                   # cleaner — replays your commits on top of main
```

Full Summary — Git Day 3

Command	What it does
`git remote add origin URL`	Connect local repo to GitHub
`git remote -v`	Show all remotes
`git push -u origin main`	Push and set upstream tracking
`git push`	Push current branch (after upstream set)
`git push origin --all`	Push all branches
`git clone URL`	Download entire repository
`git clone --depth 1 URL`	Shallow clone — latest only
`git fetch origin`	Download changes without merging
`git pull`	Fetch and merge
`git pull --rebase`	Fetch and rebase — cleaner history
`git branch -a`	Show all branches including remote
`git branch -vv`	Show tracking relationships
`git remote add upstream URL`	Add original repo (for forks)

Interview Questions — Git Day 3

Q1. What is the difference between git fetch and git pull? Answer: git fetch downloads changes from remote but does not merge them — you can review what changed before integrating. git pull does both fetch and merge in one step. git fetch is safer for production workflows — inspect first, merge when confident.

Q2. What is a Pull Request? Answer: A Pull Request is a request to merge one branch into another with a code review step. Team members review the code, leave comments and approve before it merges. PRs ensure code quality, catch bugs and provide a discussion forum for changes. Core to all professional team workflows.

Q3. What is the difference between origin and upstream? Answer: origin is your own remote repository — where you push your changes. upstream is the original repository you forked from — where you pull updates from. In a fork workflow: you pull from upstream to stay updated, push to origin for your changes, open PR from origin to upstream.

Q4. What does git clone --depth 1 do? Answer: Creates a shallow clone — downloads only the latest commit without the full history. Much faster and uses less disk space. Useful in CI/CD pipelines where you only need current code not the entire git history.

Q5. What is git push --force-with-lease and when would you use it? Answer: A safer version of force push — it checks that nobody else has pushed to the remote branch since your last fetch before overwriting. Used after rebasing a feature branch to update the remote. Never use on shared branches like main.

Q6. Your push is rejected because remote has changes you don't have. What do you do? Answer: Run git pull --rebase origin main to bring in remote changes and replay your commits on top, then git push. The --rebase flag keeps history clean by avoiding an unnecessary merge commit.

Q7. What is forking and when do you use it? Answer: Forking creates your own copy of someone else's repository on GitHub. Used when contributing to projects you don't have write access to — open source projects, company repos as a contractor. You push to your fork and open a PR to the original repository.

Q8. How does GitHub integrate with CI/CD? Answer: GitHub Actions (or Jenkins, CircleCI etc.) monitors repository events. When you push to main or merge a PR, it triggers automated workflows — running tests, building Docker images, deploying to servers. This is the core automation in DevOps — code change automatically flows to production through the pipeline.

Homework — Before Git Day 4

1. Create a GitHub account if you don't have one
2. Set up SSH key authentication with GitHub
3. Create a new repository on GitHub called `devops-scripts`
4. Push your local `git_practice` repo to GitHub
5. Create a feature branch, push it to GitHub
6. Open a Pull Request on GitHub — even if it is just for practice
7. Clone a public repository — try: `git clone git@github.com:torvalds/linux.git --depth 1`

8. Practice `git fetch` — make a change on GitHub directly (edit a file on GitHub website) then fetch and see it locally

---

Git Day 4 we learn **Advanced Git — rebase, stash, cherry-pick, reset, tags and the commands that senior engineers use daily.** This is what separates junior from senior Git knowledge in interviews. 💪

Say **"Git Day 4"** whenever you are ready!

Git Day 4

Git Day 4 — Advanced Git

Why This Matters for 10-12 LPA Roles

Days 1-3 covered what every developer knows. Today covers what **separates junior from senior** Git knowledge.

At 10-12 LPA level interviewers expect you to know:

- Why rebase is preferred over merge in many teams
- How to recover from mistakes without panicking
- How to manage releases with tags
- How to move specific commits between branches

These commands also come up in real daily work constantly.

What You Will Learn Today

- `git stash` — save work without committing
- `git rebase` — rewrite history cleanly
- `git cherry-pick` — take specific commits
- `git reset` — undo commits
- `git revert` — safe undo for shared branches
- `git reflog` — recover anything
- `git tag` — mark releases
- `git bisect` — find which commit broke something
- `.git` internals — what is actually happening

`git stash` — Save Work Without Committing

The Problem

You are halfway through fixing a bug. Your manager calls — critical issue on production needs immediate attention. You cannot commit half-done work. You cannot lose your changes.

`git stash` saves your work temporarily so you can switch context.

Basic Stash Commands

```bash
git stash                       # save all uncommitted changes
git stash push -m "half-done login fix"   # save with a description
git stash list                  # see all stashes
git stash pop                   # restore latest stash AND remove from list
git stash apply                 # restore latest stash BUT keep in list
git stash apply stash@{2}       # restore specific stash by index
git stash drop stash@{0}        # delete specific stash
git stash clear                 # delete ALL stashes
git stash show -p               # show what is in the stash
```

`stash pop` vs `stash apply`

```bash
git stash pop       # restore AND remove from stash list — use when done
git stash apply     # restore BUT keep in stash list — use if you want to
                    # apply same changes to multiple branches
```

Real Scenario — Interrupted Work

```bash
# You are working on feature
vim deploy.sh          # half done changes
# Manager calls — production is down!

git stash push -m "deploy script — half done"
git switch main
git switch -c hotfix/production-crash
```

```bash
# Fix the production issue
vim auth.sh
git add auth.sh
git commit -m "Fix auth crash on missing token"
git push origin hotfix/production-crash
# PR merged — production fixed

# Back to your original work
git switch feature/deploy-improvements
git stash pop          # your half-done work is back
# Continue where you left off
```

Stashing Untracked Files

bash

```
git stash                    # only stashes tracked files
git stash -u                 # also stashes untracked new files
git stash -a                 # stashes everything including .gitignored files
```

⚠️ Stash is Local Only

Stashes are stored in `.git/refs/stash` — they are not pushed to GitHub. If you dele

---

Interview Question

Q. What is git stash and when would you use it?
Answer: git stash temporarily saves uncommitted changes so you can switch to another

---

`git rebase` — Rewrite History Cleanly

The Problem With Merge

When you merge main into your feature branch repeatedly you get a messy history full
```
feature:  A---B---C---M---D---E---M---F
                  |           |
main:     A---B---C---G---H---I
```

```
```

`M` commits are merge commits — they clutter the history and make it hard to underst

---

What Rebase Does

Rebase **replays your commits on top of another branch** — as if you started your fe
```
Before rebase:
main:    A---B---C---D---E
              \
feature:       F---G

After: git rebase main (from feature branch)
main:    A---B---C---D---E
                          \
feature:                   F'---G'
```

F and G are replayed as F' and G' on top of E. Clean linear history. No merge commits.

Basic Rebase

```bash
# Update your feature branch with latest main — cleanly:
git switch feature/my-feature
git rebase main

# If conflicts during rebase:
# Fix the conflict in the file
git add conflicting-file.txt
git rebase --continue      # continue to next commit

# If you want to give up:
git rebase --abort         # goes back to state before rebase started
```

Merge vs Rebase — Side by Side

```bash
# Merge approach — preserves full history:
git switch feature
git merge main
# Creates merge commit — non-destructive, safer
```

```
# Rebase approach — clean linear history:
git switch feature
git rebase main
# Rewrites commits — cleaner but changes commit SHAs
```

Merge	Rebase
Preserves complete history	Creates clean linear history
Safe on shared branches	Never rebase shared branches
Creates merge commits	No merge commits
Non-destructive	Rewrites commit history
Easier to understand	Preferred by many teams

The Golden Rule of Rebase

Never rebase commits that exist on a public/shared branch.

```bash
# SAFE — rebasing your local feature branch:
git switch feature/my-feature
git rebase main               # OK — nobody else has this branch

# DANGEROUS — rebasing main:
git switch main
git rebase feature            # NEVER DO THIS — main is shared
```

When you rebase, commits get new SHAs. If someone else pulled the old SHAs — their history now conflicts badly.

Interactive Rebase — Powerful Cleanup

```bash
git rebase -i HEAD~3    # interactively edit last 3 commits
git rebase -i HEAD~5    # interactively edit last 5 commits
```

Opens editor showing your recent commits:

```
pick a1b2c3d Add monitoring script
pick d4e5f6g Fix typo in monitoring script
pick h7i8j9k Add another typo fix
```

Change the word `pick` to:

```bash
pick    # keep commit as-is
reword  # keep commit but edit the message
edit    # pause and amend this commit
squash  # combine with previous commit
fixup   # combine with previous — discard this commit message
drop    # delete this commit entirely
```

Squashing Commits — Very Common in Teams

Many teams require you to squash all your feature commits into one clean commit befo
```
Before squash:
d4e5f6g Fix typo
a1b2c3d Fix another typo
z9y8x7w Actually fix the bug
h7i8j9k Add monitoring script

After squash into one:
a1b2c3d Add nginx monitoring script with error handling
```

```bash
git rebase -i HEAD~4    # edit last 4 commits

# In the editor:
pick h7i8j9k Add monitoring script
squash z9y8x7w Actually fix the bug
squash a1b2c3d Fix another typo
squash d4e5f6g Fix typo

# Save — Git combines all into one commit
# Edit the final commit message to be clean and clear
```

Interview Question

Q. What is the difference between git merge and git rebase?
Answer: Both integrate changes from one branch into another. Merge preserves complet

```
---

# `git cherry-pick` — Take Specific Commits

---

## What is Cherry-Pick?

Cherry-pick applies a **specific commit from any branch** onto your current branch.
```
main: A---B---C---D---E
              \
feature: F---G---H

You want only commit G on main — not F or H
git switch main
git cherry-pick G_SHA

main: A---B---C---D---E---G'
```

---

### Basic Cherry-Pick

```bash
# Find the SHA of the commit you want:
git log --oneline feature-branch

# Apply it to current branch:
git cherry-pick a1b2c3d              # apply one commit
git cherry-pick a1b2c3d d4e5f6g      # apply multiple commits
git cherry-pick a1b2c3d..h7i8j9k    # apply a range of commits
git cherry-pick -n a1b2c3d           # apply changes but don't commit yet
```

Real DevOps Scenario — Backport a Fix

You have two branches: `main` (latest) and `release-v1` (older version still in production).

You fixed a critical security bug on main. You need that SAME fix on `release-v1` without merging all other changes.

```bash
# On main — find the fix commit SHA
git log --oneline main
# a1b2c3d Fix SQL injection in user auth
```
```

```bash
# Switch to release branch
git switch release-v1

# Cherry-pick just that fix
git cherry-pick a1b2c3d

# Push to deploy the fix
git push origin release-v1
```

This is called **backporting** — applying a fix to older releases. Very common in real DevOps work.

Cherry-Pick Conflicts

```bash
git cherry-pick a1b2c3d
# CONFLICT — same as merge conflicts

# Fix the conflict in the file
git add conflicting-file.txt
git cherry-pick --continue     # apply the cherry-pick with fix
# or
git cherry-pick --abort        # cancel and go back
```

Interview Question

Q. What is git cherry-pick and when would you use it?
Answer: Cherry-pick applies a specific commit from one branch onto another without m

---

`git reset` — Undo Commits

What is Reset?

Reset moves the branch pointer backwards — effectively undoing commits.
```
Before reset:
A---B---C---D  (HEAD, main)

After git reset HEAD~2:
```

```
A---B          (HEAD, main)
    C and D are gone from branch history
```

Three Types of Reset

```bash
git reset --soft HEAD~1     # undo commit — keep changes STAGED
git reset --mixed HEAD~1    # undo commit — keep changes in WORKING DIR (default)
git reset --hard HEAD~1     # undo commit — DELETE changes completely
```

Flag	Commit	Staging Area	Working Dir
`--soft`	Undone	Changes kept staged	Unchanged
`--mixed`	Undone	Cleared	Changes kept
`--hard`	Undone	Cleared	Changes DELETED

When to Use Each

```bash
# --soft: undo commit but keep changes staged — want to recommit differently
git reset --soft HEAD~1
# Now files are staged — just commit again with better message

# --mixed (default): undo commit, unstage changes — most common
git reset HEAD~1
# Changes are in working dir — review, re-stage what you want

# --hard: completely undo — discard all changes
git reset --hard HEAD~1
# Changes are GONE — use with extreme caution
```

Reset with Specific SHA

```bash
git log --oneline
# a1b2c3d Latest feature
# d4e5f6g Good stable state
# h7i8j9k Initial commit

# Reset to specific point:
```

```bash
git reset --hard d4e5f6g
# Now at d4e5f6g — everything after it is gone from history
```

⚠️ Reset vs Revert

```bash
# reset — rewrites history — NEVER use on shared branches
git reset HEAD~1

# revert — creates new commit that undoes changes — SAFE for shared branches
git revert HEAD
```

`git revert` — Safe Undo for Shared Branches

What is Revert?

Revert creates a **new commit that undoes a previous commit.** History is preserved

```
Before revert:
A---B---C---D  (bad commit D)

After git revert D:
A---B---C---D---D'
D' undoes what D did — D still exists in history
```

Basic Revert

```bash
git revert HEAD              # revert last commit
git revert a1b2c3d           # revert specific commit
git revert HEAD~3..HEAD      # revert last 3 commits
git revert --no-commit HEAD  # stage the revert without committing
```

Reset vs Revert — The Critical Difference

```bash
```

```bash
# Use reset when:
# → You have NOT pushed yet — local only
# → You want to completely undo
git reset --hard HEAD~1

# Use revert when:
# → You HAVE pushed to shared branch
# → You need to undo while preserving history
git revert a1b2c3d
git push origin main    # safe — history preserved
```

Rule: Reset for local, Revert for shared/remote.

Real Scenario — Bad Deployment

```bash
# Bad code was merged and pushed to main
git log --oneline
# a1b2c3d Deploy broken feature    ← this broke production
# d4e5f6g Previous stable release

# Cannot reset — other people have pulled main
# Use revert:
git revert a1b2c3d
# Creates new commit that undoes the broken changes
git push origin main    # production is fixed, history preserved
```

Interview Question

Q. What is the difference between git reset and git revert? Answer: git reset moves the branch pointer backwards rewriting history — only safe for local commits not yet pushed. git revert creates a new commit that undoes the changes of a previous commit while preserving history — safe for shared branches that others have pulled. Rule: reset locally, revert remotely.

`git reflog` — Recover Anything

What is Reflog?

Reflog records every movement of HEAD — every commit, checkout, reset, merge, rebase. It is Git's safety net. Even after `git reset --hard` — your work is recoverable with reflog.

```bash
```

```
git reflog              # show all HEAD movements
git reflog show main    # show movements of main branch
```

Output:
```
a1b2c3d HEAD@{0}: commit: Add monitoring script
d4e5f6g HEAD@{1}: checkout: moving from main to feature
h7i8j9k HEAD@{2}: reset: moving to HEAD~1
z9y8x7w HEAD@{3}: commit: Previous good commit
```

Recovering Lost Commits

```bash
# You ran git reset --hard and lost your work
git reset --hard HEAD~3    # oops — lost 3 commits

# Find the lost commits in reflog:
git reflog
# a1b2c3d HEAD@{0}: reset: moving to HEAD~3
# d4e5f6g HEAD@{1}: commit: Lost commit 3    ← this is what you want

# Recover by creating branch at that point:
git switch -c recovery-branch d4e5f6g

# Or reset back to that point:
git reset --hard d4e5f6g    # back to before the accidental reset
```

Reflog Expiry

Reflog entries expire after 90 days by default. So you have 90 days to recover anyth

---

Interview Question

Q. How do you recover commits after git reset --hard?
Answer: Using git reflog — it records every HEAD movement including resets. Find the

---

`git tag` — Marking Releases

```
---

## What are Tags?

Tags mark specific commits as important — usually software releases. Unlike branches
```
main: A---B---C---D---E---F
                  | |
               v1.0.0   v1.1.0
```

---

### Two Types of Tags

```bash
# Lightweight tag — just a pointer to commit:
git tag v1.0.0

# Annotated tag — has metadata — PREFERRED for releases:
git tag -a v1.0.0 -m "Release version 1.0.0 — initial production release"
```

Always use annotated tags for releases — they contain tagger name, date and message.

Tag Commands

```bash
git tag                              # list all tags
git tag -l "v1.*"                    # list tags matching pattern
git tag -a v1.0.0 -m "Release v1.0" # create annotated tag
git tag -a v1.0.0 a1b2c3d -m "msg"  # tag specific past commit
git show v1.0.0                      # show tag details
git push origin v1.0.0               # push specific tag to GitHub
git push origin --tags               # push all tags to GitHub
git tag -d v1.0.0                    # delete tag locally
git push origin --delete v1.0.0      # delete tag on remote
```

Semantic Versioning — What the Numbers Mean
```
v1.2.3
  | | |
  | | └── Patch — bug fixes only (backwards compatible)
```
```

```
  | └──── Minor — new features (backwards compatible)
  └────── Major — breaking changes (not backwards compatible)
```

```bash
git tag -a v1.0.0 -m "First stable release"
git tag -a v1.0.1 -m "Fix login crash bug"       # patch
git tag -a v1.1.0 -m "Add monitoring dashboard"   # minor feature
git tag -a v2.0.0 -m "Rewrite authentication"     # major — breaking
```

Checking Out a Tag

```bash
git checkout v1.0.0    # go to exact state of v1.0.0
# You are in detached HEAD state — not on any branch
# To make changes:
git switch -c fix-v1.0.0    # create branch from this tag
```

Real DevOps Use

```bash
# Release workflow:
git switch main
git pull

# Run tests — all passing
# Create release tag
git tag -a v2.1.0 -m "Release v2.1.0 — add backup automation"
git push origin v2.1.0

# CI/CD pipeline detects new tag → automatically builds Docker image
# → deploys to production
```

Tags trigger CI/CD pipelines in most setups. When you push a `v*` tag — the pipeline automatically builds and deploys. You will implement this when you reach CI/CD.

`git bisect` — Find the Breaking Commit

What is Bisect?

Bisect uses **binary search** to find which commit introduced a bug. Instead of checking 100 commits one by one — bisect narrows it down in 7 steps.

How Bisect Works

```bash
git bisect start                # start bisect session
git bisect bad                  # current commit is broken
git bisect good v1.0.0          # v1.0.0 was working

# Git checks out a commit halfway between bad and good
# You test — is this commit good or bad?
git bisect good                 # this commit works
# or
git bisect bad                  # this commit is broken

# Git narrows down — repeat until found:
# Output: a1b2c3d is the first bad commit

git bisect reset                # exit bisect — back to original branch
```

Automated Bisect

```bash
# If you have a test script:
git bisect start
git bisect bad HEAD
git bisect good v1.0.0
git bisect run ./test.sh    # runs test automatically on each commit
# Git finds the bad commit automatically
git bisect reset
```

Real Scenario

Production broke sometime in the last week. You have 50 commits. Bisect finds the culprit in 6 steps instead of 50.

`.git` Internals — What is Actually Happening

Understanding What Git Stores

```bash
```

```
ls .git/
```

```
HEAD            # pointer to current branch
config          # repo config
objects/        # ALL your file content stored here
refs/           # branches and tags
hooks/          # scripts triggered by git events
COMMIT_EDITMSG  # last commit message
index           # staging area data
```

Git Object Types

Git stores everything as objects in `.git/objects/`:

```bash
# 4 types of objects:
blob    # file content
tree    # directory structure
commit  # commit metadata + pointer to tree
tag     # annotated tag object
```

```bash
# See any object:
git cat-file -t a1b2c3d    # type of object
git cat-file -p a1b2c3d    # contents of object
```

Git Hooks — Automation at Every Step

Hooks are scripts in `.git/hooks/` that run automatically on git events:

```bash
ls .git/hooks/
# pre-commit        — runs before every commit
# commit-msg        — validates commit message format
# pre-push          — runs before push
# post-merge        — runs after merge
# post-checkout     — runs after checkout
```

Example — Enforce Commit Message Format

```bash
vim .git/hooks/commit-msg
```

```bash
#!/bin/bash
# Enforce conventional commits format
commit_msg=$(cat "$1")
pattern="^(feat|fix|docs|style|refactor|test|chore): .+"

if ! echo "$commit_msg" | grep -qE "$pattern"; then
    echo "ERROR: Commit message must follow format:"
    echo "  feat: add new feature"
    echo "  fix: fix a bug"
    echo "  docs: update documentation"
    exit 1
fi
```

```bash
chmod +x .git/hooks/commit-msg

# Now bad commit messages are rejected:
git commit -m "stuff"
# ERROR: Commit message must follow format
```

Example — Run Tests Before Push

```bash
vim .git/hooks/pre-push
```

```bash
#!/bin/bash
echo "Running tests before push..."
./run_tests.sh
if [ $? -ne 0 ]; then
    echo "Tests failed — push blocked"
    exit 1
fi
echo "Tests passed — pushing"
```

```bash
```

```bash
chmod +x .git/hooks/pre-push
```

Now broken code can never be pushed — tests must pass first. This is basic CI enforcement at the local level.

Complete Advanced Git Cheat Sheet

Stash

```bash
git stash                       # save work
git stash push -m "description" # save with message
git stash list                   # list stashes
git stash pop                    # restore and remove
git stash apply stash@{1}        # restore specific
git stash clear                  # delete all
```

Rebase

```bash
git rebase main                  # rebase onto main
git rebase -i HEAD~3             # interactive — edit last 3
git rebase --continue            # after resolving conflict
git rebase --abort               # cancel rebase
```

Cherry-Pick

```bash
git cherry-pick SHA              # apply one commit
git cherry-pick SHA1 SHA2        # apply multiple
git cherry-pick SHA1..SHA2       # apply range
git cherry-pick --abort          # cancel
```

Reset and Revert

```bash
git reset --soft HEAD~1          # undo commit keep staged
git reset HEAD~1                 # undo commit keep in workdir
```

```bash
git reset --hard HEAD~1          # undo commit delete changes
git revert HEAD                  # safe undo — new commit
git revert SHA                   # revert specific commit
```

Reflog and Recovery

```bash
git reflog                       # see all HEAD movements
git reset --hard SHA             # recover to specific point
git switch -c recovery SHA       # recover on new branch
```

Tags

```bash
git tag -a v1.0.0 -m "msg"     # create annotated tag
git push origin --tags          # push all tags
git tag -d v1.0.0               # delete local tag
git push origin --delete v1.0.0 # delete remote tag
```

Full Interview Questions — Git Day 4

Q1. What is git stash and when would you use it? Answer: git stash temporarily saves uncommitted changes so you can switch branches without committing incomplete work. Used when you need to urgently switch context — like a production hotfix interrupting feature work. `git stash pop` restores the work when you return.

Q2. What is the difference between git merge and git rebase? Answer: Merge integrates branches by creating a merge commit preserving full history — safe and non-destructive. Rebase replays commits on top of target branch creating clean linear history but rewrites SHAs. Never rebase commits on shared branches that others have pulled.

Q3. What is interactive rebase used for? Answer: `git rebase -i HEAD~N` lets you edit, reorder, squash or drop recent commits before pushing. Used to clean up messy commit history — squash multiple small commits into one clean commit, fix commit messages, remove debug commits before opening a PR.

Q4. What is git cherry-pick? Answer: Applies a specific commit from one branch onto another without merging the entire branch. Used for backporting — applying a critical fix to an older release branch. Also used when you accidentally committed to the wrong branch.

Q5. What is the difference between git reset and git revert? Answer: git reset moves branch pointer backwards rewriting history — only safe for local commits not pushed. git

revert creates a new commit undoing changes while preserving history — safe for shared branches. Rule: reset locally, revert remotely.

Q6. How do you recover commits after git reset --hard? Answer: Using git reflog — it records every HEAD movement. Find the SHA of your lost commits then `git reset --hard that_SHA` to restore, or `git switch -c recovery that_SHA` to recover on a new branch. Works within 90 days before reflog entries expire.

Q7. What are Git tags and what is the difference between lightweight and annotated? Answer: Tags mark specific commits permanently — usually for releases. Lightweight tags are just pointers to commits. Annotated tags contain metadata — tagger name, date, message — and are preferred for official releases. Use semantic versioning: v1.2.3 — major.minor.patch.

Q8. What are Git hooks? Answer: Scripts in .git/hooks/ that run automatically on git events. pre-commit runs before every commit, commit-msg validates commit message format, pre-push runs before pushing. Used to enforce code standards, run tests and prevent bad commits locally before they reach the remote.

Q9. What is git bisect? Answer: Uses binary search to find which commit introduced a bug. You tell it a good commit and a bad commit — Git checks out the midpoint, you test and mark good or bad, repeating until the exact breaking commit is found. Reduces 100 commit investigation to 7 steps.

Q10. What does git reflog show? Answer: Every movement of HEAD — every commit, checkout, reset, merge, rebase. It is Git's safety net. Even commits deleted with git reset --hard are recoverable through reflog within 90 days.

Homework — Consolidate Everything

bash

```
cd ~/linux_practice/git_practice
```

```
1. Make 3 messy commits with bad messages — then use `git rebase -i HEAD~3` to squas
2. Create a stash, switch branches, come back and pop the stash
3. Find a commit SHA from `git log --oneline` and cherry-pick it to a new branch
4. Make a commit then `git reset --soft HEAD~1` — see that changes are still staged
5. Make a commit then `git revert HEAD` — see that it creates a new commit
6. Run `git reflog` — understand every line in the output
7. Create an annotated tag `v1.0.0` and push it to GitHub
8. Create a `pre-commit` hook that prevents commits with the message "WIP"

---

## 🎉 Git Training Complete!
```

```
Git Day 1 ✅  init, add, commit, log, .gitignore
```

```
Git Day 2 ✅  branches, merging, conflicts
Git Day 3 ✅  GitHub, push, pull, clone, PRs
Git Day 4 ✅  stash, rebase, cherry-pick, reset, revert, reflog, tags
```

Your Complete Roadmap Status

```
Linux  ━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE — 16 days
Git    ━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE — 4 days
Docker ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  ← NEXT
AWS    ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  (doing in parallel chat)
CI/CD  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  After Docker
K8s    ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  After CI/CD
```

Say **"update document"** to pack everything into your notes, or say **"Docker Day 1"** to start Docker training! 💪

No make a separate document of Git and git hub and , Docker Day 1

Let me do both simultaneously — Git document first, then Docker Day 1!

Good. Now let me build the Git document first, then deliver Docker Day 1.

Git document ready! ✅ Now let's start Docker Day 1!

Docker Day 1 — Introduction & Core Concepts

Why Docker is Non-Negotiable for DevOps

Here is the most common problem in software development before Docker:

"It works on my machine."

Developer builds an app on their laptop — works perfectly. They deploy to server — crashes immediately. The server has a different version of Python, different libraries, different OS settings.

Docker solves this completely. You package your application with everything it needs — code, runtime, libraries, config — into one unit called a **container.** It runs identically everywhere.

Docker is in every DevOps job description. No exceptions.